

CPU Scheduling Algorithms

FCFS, SJF, SRTF, Round Robin and Priority Scheduling

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

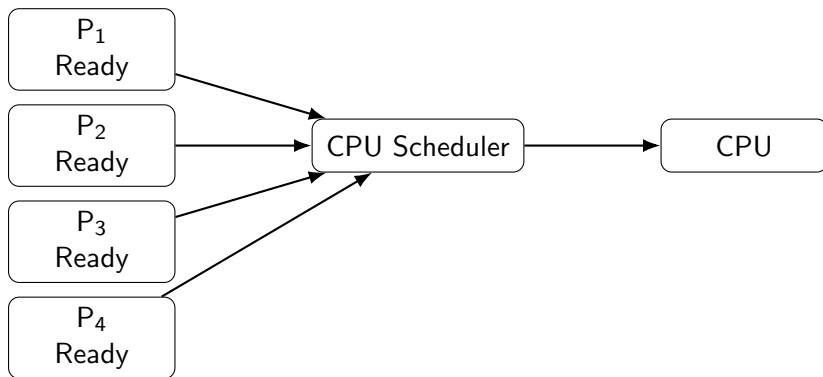
Learning Objectives

By the end of this topic, students should be able to:

- Explain the operation of FCFS, SJF, SRTF, Round Robin and Priority scheduling.
- Distinguish preemptive and non-preemptive scheduling.
- Construct Gantt charts for different scheduling policies.
- Calculate:
 - Completion Time (CT)
 - Turnaround Time (TAT)
 - Waiting Time (WT)
 - Response Time (RT)
- Calculate average scheduling performance.
- Analyze starvation, convoy effect, fairness and context-switch overhead.
- Select a scheduling policy according to workload characteristics.

CPU Scheduling: Basic Idea

Several processes can be ready at the same time, but one CPU core can execute only one process at a time.



Scheduling decision

The scheduler selects which ready process receives the CPU next.

Process Parameters Used in Scheduling Problems

For each process P_i :

Symbol	Meaning
AT	Arrival Time – when process enters the ready queue
BT	Burst Time – CPU execution time required
ST	First Start Time – first time process gets CPU
CT	Completion Time – time at which process finishes
TAT	Turnaround Time
WT	Waiting Time
RT	Response Time

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

Performance Metrics

Turnaround Time

$$TAT = \text{Completion Time} - \text{Arrival Time}$$

Total time spent in the system.

Waiting Time

$$WT = TAT - \text{CPU Burst Time}$$

Total time spent waiting in the ready queue.

Response Time

$$RT = \text{First CPU Start Time} - \text{Arrival Time}$$

Important for interactive systems.

Preemptive vs. Non-Preemptive Scheduling

Non-Preemptive

A running process keeps the CPU until:

- it terminates, or
- it blocks for I/O.

Examples:

- FCFS
- Non-preemptive SJF
- Non-preemptive Priority

Preemptive

The OS can interrupt a running process and select another ready process.

Examples:

- SRTF
- Round Robin
- Preemptive Priority

Trade-off

Preemption can improve response time, but context switches add overhead.

Common Example Dataset

The following process set will be used to demonstrate the algorithms.

Process	Arrival Time	Burst Time	Priority
P ₁	0	8	2
P ₂	1	4	1
P ₃	2	2	3
P ₄	3	1	4

- Smaller priority number = higher priority.
- All times are measured in time units.
- Assume one CPU and zero context-switch cost for the basic calculations.

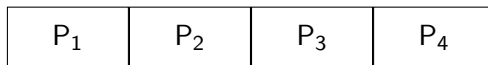
Why one common dataset?

Using the same processes makes the effect of the scheduling policy easier to compare.

Algorithm 1: FCFS

First-Come, First-Served

- Processes execute in order of arrival.
- Normally non-preemptive.
- Simple FIFO-like policy.
- Once a process starts, it normally runs until completion or blocking.



—————→
arrival order

Main issue

A long CPU-bound process can make many short processes wait behind it: the **convoy effect**.

FCFS Example: Gantt Chart

Dataset:

$$P_1(AT = 0, BT = 8), \quad P_2(AT = 1, BT = 4), \quad P_3(AT = 2, BT = 2), \quad P_4(AT = 3, BT = 1)$$

FCFS order:

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$$



Completion times:

$$CT_1 = 8, \quad CT_2 = 12, \quad CT_3 = 14, \quad CT_4 = 15$$

FCFS: Calculate TAT, WT and RT

P	AT	BT	ST	CT	TAT	WT	RT
P ₁	0	8	0	8	8 - 0 = 8	8 - 8 = 0	0 - 0 = 0
P ₂	1	4	8	12	12 - 1 = 11	11 - 4 = 7	8 - 1 = 7
P ₃	2	2	12	14	14 - 2 = 12	12 - 2 = 10	12 - 2 = 10
P ₄	3	1	14	15	15 - 3 = 12	12 - 1 = 11	14 - 3 = 11

$$Avg.WT = \frac{0 + 7 + 10 + 11}{4} = \boxed{7}$$

$$Avg.TAT = \frac{8 + 11 + 12 + 12}{4} = \boxed{10.75}$$

$$Avg.RT = \frac{0 + 7 + 10 + 11}{4} = \boxed{7}$$

Observation

The short processes P₃ and P₄ wait behind the long P₁.

FCFS: Convoy Effect

Consider:

$$P_1 : BT = 20, \quad P_2 : BT = 2, \quad P_3 : BT = 2$$

If P_1 arrives first:



P_1

P_2 and P_3 are short jobs but experience very large waiting times.

Real-world analogy

A single customer with a very large order at a counter can delay many customers with small orders.

Algorithm 2: SJF

Shortest Job First

- Select the ready process with the smallest CPU burst.
- Basic SJF is non-preemptive.
- If the CPU is idle, choose among currently available processes.
- When a process starts, it normally runs to completion.

Key idea

If short jobs are executed first, average waiting time can be reduced substantially.

Practical difficulty

The exact future CPU burst length is generally unknown; operating systems estimate it using past behavior.

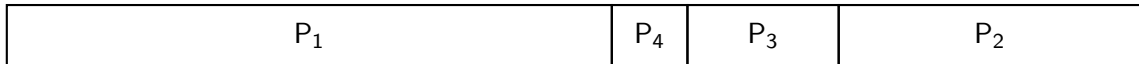
SJF Example: Scheduling Decision

Using the common dataset:

Time	Ready Processes	Decision
0	P ₁	P ₁ starts
1	P ₂	P ₁ continues
2	P ₂ , P ₃	P ₁ continues
3	P ₂ , P ₃ , P ₄	P ₁ continues
8	P ₂ , P ₃ , P ₄	P ₄ (BT=1)
9	P ₂ , P ₃	P ₃ (BT=2)
11	P ₂	P ₂ (BT=4)

Therefore:

$$P_1 \rightarrow P_4 \rightarrow P_3 \rightarrow P_2$$



SJF: Calculate TAT, WT and RT

P	AT	BT	ST	CT	TAT	WT	RT
P ₁	0	8	0	8	8	0	0
P ₂	1	4	11	15	14	10	10
P ₃	2	2	9	11	9	7	7
P ₄	3	1	8	9	6	5	5

$$Avg.WT = \frac{0 + 10 + 7 + 5}{4} = \boxed{5.5}$$

$$Avg.TAT = \frac{8 + 14 + 9 + 6}{4} = \boxed{9.25}$$

$$Avg.RT = \frac{0 + 10 + 7 + 5}{4} = \boxed{5.5}$$

Important

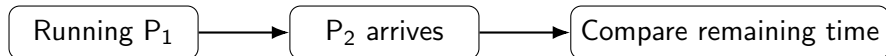
SJF improves the average for this workload compared with FCFS, but a long process may wait for a long time.

Algorithm 3: SRTF

Shortest Remaining Time First

SRTF is the preemptive form of SJF.

- At every scheduling decision, choose the ready process with the shortest remaining CPU time.
- A newly arriving process can preempt the running process.
- It can improve response and average waiting time.
- It may cause more context switches.



Decision rule

Run the process with minimum remaining CPU burst.

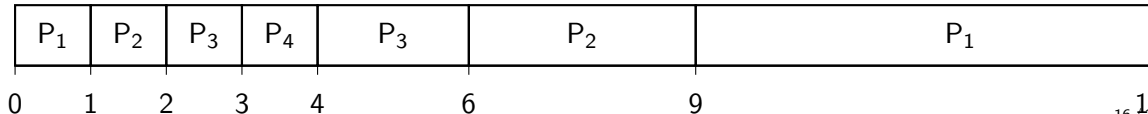
SRTF Example: Step-by-Step Scheduling

Common dataset:

$P_1(AT = 0, BT = 8), P_2(AT = 1, BT = 4), P_3(AT = 2, BT = 2), P_4(AT = 3, BT = 1)$

Scheduling decisions:

- $t = 0$: P_1 starts; remaining time = 8.
- $t = 1$: P_2 arrives; P_1 has 7, so P_2 preempts P_1 .
- $t = 2$: P_3 arrives; P_2 has 3, P_3 has 2 $\Rightarrow P_3$ runs.
- $t = 3$: P_4 arrives; P_4 has 1, so P_4 runs.
- $t = 4$: P_3 resumes and finishes at $t = 6$.
- P_2 then runs from 6 to 9.
- P_1 finally runs from 9 to 16.



SRTF: Calculate Performance Metrics

P	AT	BT	ST	CT	TAT	WT	RT
P ₁	0	8	0	16	16	8	0
P ₂	1	4	1	9	8	4	0
P ₃	2	2	2	6	4	2	0
P ₄	3	1	3	4	1	0	0

$$Avg.WT = \frac{8 + 4 + 2 + 0}{4} = \boxed{3.5}$$

$$Avg.TAT = \frac{16 + 8 + 4 + 1}{4} = \boxed{7.25}$$

$$Avg.RT = \frac{0 + 0 + 0 + 0}{4} = \boxed{0}$$

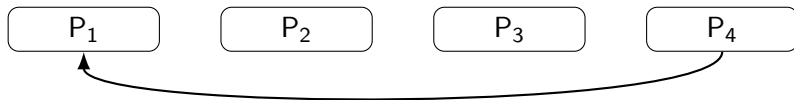
Observation

SRTF gives very good response for short arriving processes in this example, but P₁ is repeatedly delayed.

Algorithm 4: Round Robin

Round Robin (RR)

- Designed for time-sharing and interactive systems.
- Each ready process receives a fixed time quantum.
- If it does not finish within the quantum, it is preempted.
- The process moves to the end of the ready queue.



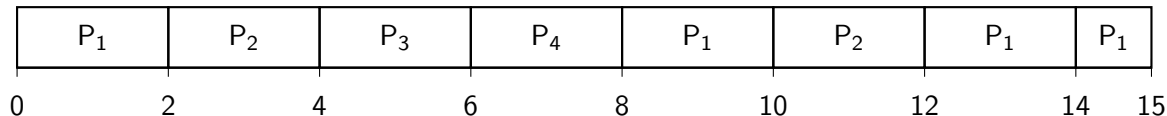
Choose next process in circular FIFO order

Critical parameter

The **time quantum** q strongly affects response time, fairness and context-switch overhead.

Round Robin Example: Quantum $q = 2$

Use the same process set and $q = 2$.



Completion times:

$$CT_3 = 6, \quad CT_4 = 8, \quad CT_2 = 12, \quad CT_1 = 15$$

Important

RR scheduling must account for processes that arrive while another process is executing. The ready-queue order at each quantum boundary matters.

Round Robin: Correct Metric Calculation

For the above schedule:

P	AT	BT	ST	CT	TAT	WT	RT
P ₁	0	8	0	15	15	7	0
P ₂	1	4	2	12	11	7	1
P ₃	2	2	4	6	4	2	2
P ₄	3	1	6	8	5	4	3

$$Avg.WT = \frac{7 + 7 + 2 + 4}{4} = \boxed{5}$$

$$Avg.TAT = \frac{15 + 11 + 4 + 5}{4} = \boxed{8.75}$$

$$Avg.RT = \frac{0 + 1 + 2 + 3}{4} = \boxed{1.5}$$

Interpretation

Relatively fast initial response while giving each ready process repeated CPU opportunities. 20 / 38

Effect of Time Quantum in Round Robin

q too small

- More context switches
- Higher scheduling overhead
- Less useful CPU time
- Usually better responsiveness

q too large

- Fewer context switches
- Lower overhead
- RR approaches FCFS behavior
- Interactive response can worsen

Choose q large enough to control overhead but small enough for responsiveness

Algorithm 5: Priority Scheduling

Each process is assigned a priority.

- The highest-priority ready process is selected.
- Priority may be static or dynamic.
- Can be preemptive or non-preemptive.
- In this example, **smaller number = higher priority**.

Example:

$$P_1 = 2, \quad P_2 = 1, \quad P_3 = 3, \quad P_4 = 4$$

Therefore:

$$P_2 > P_1 > P_3 > P_4$$

Important

Priority numbers have no universal meaning. Always state whether a smaller or larger number represents higher priority.

Non-Preemptive Priority Example

Using the common dataset:

At $t = 0$, only P_1 is available, so P_1 runs until $t = 8$.

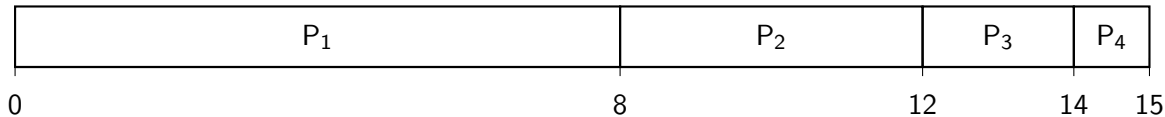
At $t = 8$, ready processes are:

$P_2(Priority = 1)$, $P_3(Priority = 3)$, $P_4(Priority = 4)$

Thus:

$P_2 \rightarrow P_3 \rightarrow P_4$

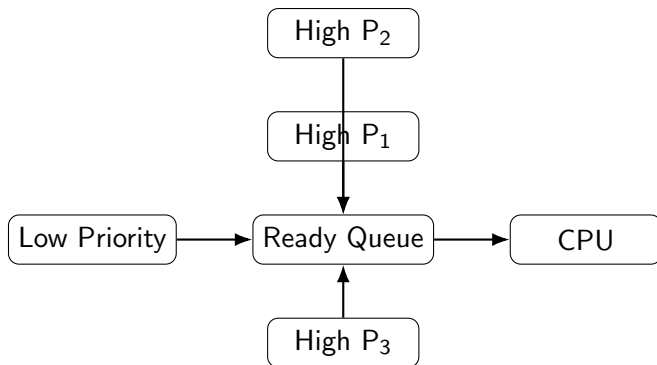
Gantt chart:



$$Avg. WT = \frac{0 + 7 + 10 + 11}{4} = \boxed{7}$$

Priority Scheduling: Starvation

A low-priority process may wait indefinitely if high-priority processes continuously arrive.



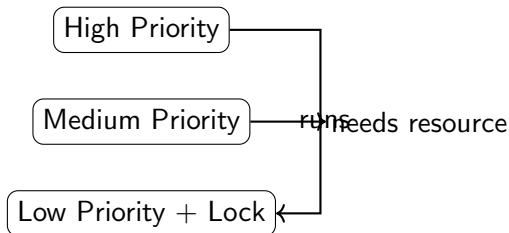
Solution: Aging

Gradually increase the priority of a process as it waits for a long time.

Priority Inversion

Priority inversion can occur when:

- 1 A high-priority process needs a resource held by a low-priority process.
- 2 A medium-priority process keeps running.
- 3 The high-priority process remains blocked.



Common mitigation:

- Priority inheritance
- Priority ceiling protocols

Comparison of Scheduling Algorithms

Algorithm	Preemption	Main Strength	Main Concern	Typical Use
FCFS	No	Very simple	Convoy effect	Simple batch workloads
SJF	No	Low average WT under ideal burst knowledge	Burst prediction; starvation risk	Batch workloads
SRTF	Yes	Excellent short-job responsiveness	More context switches; starvation	Short-job oriented workloads
RR	Yes	Fairness and responsiveness	Quantum selection	Time-sharing systems
Priority	Either	Supports importance levels	Starvation / inversion	Priority-sensitive systems

Key principle

The “best” algorithm depends on the system objective and workload.

Performance Comparison for the Common Dataset

Algorithm	Avg. WT	Avg. TAT	Avg. RT
FCFS	7.00	10.75	7.00
SJF	5.50	9.25	5.50
SRTF	3.50	7.25	0.00
RR ($q = 2$)	5.00	8.75	1.50
Priority (non-preemptive)	7.00	10.75	7.00

Do not memorize the ranking

These values are specific to this process set. A different arrival pattern, burst distribution, priority assignment or quantum can produce different results.

How to Solve Any CPU Scheduling Problem

Use this systematic procedure:

- 1 Write all processes with AT, BT and priority if applicable.
- 2 Identify whether the algorithm is preemptive.
- 3 At each decision point, list the ready processes.
- 4 Apply the algorithm's selection rule.
- 5 Draw the Gantt chart.
- 6 Determine each process's first start time and completion time.
- 7 Calculate:

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

- 8 Calculate averages.
- 9 Interpret the result using workload characteristics.

Common Mistakes in Scheduling Calculations

- Using completion time directly as turnaround time when $AT \neq 0$.
- Confusing waiting time with response time.
- Forgetting newly arrived processes in SRTF.
- Treating SJF as preemptive; its preemptive counterpart is SRTF.
- Ignoring the time quantum in Round Robin.
- Forgetting that a process may execute in multiple segments under RR/SRTF.
- Using the wrong priority direction.
- Ignoring idle CPU intervals.
- Forgetting context-switch overhead when a question explicitly includes it.

Best practice

Draw the Gantt chart first. Then calculate CT, TAT, WT and RT from the chart.

Real-World Interpretation

FCFS

Like a normal queue at a service counter.
Simple and predictable, but one long request can delay everyone.

SJF/SRTF

Like processing small jobs first.
Useful when short jobs are important and burst estimates are available.

RR

Like giving every person a fixed amount of service time before moving to the next person.
Useful for interactive sharing.

Priority

Like an emergency department: urgency can determine service order, but starvation and fairness must be controlled.

Workload-Based Selection

Workload	Important Objective	Possible Scheduling Approach
Batch processing	Throughput / average turnaround	SJF-family approaches
Interactive desktop	Response time / fairness	Round Robin or feedback-based approaches
Priority-sensitive work	Respect importance levels	Priority scheduling
Simple sequential workload	Low implementation complexity	FCFS
Short interactive jobs	Fast response	SRTF/RR depending on workload

Analytical Question 1

Four processes are given:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	1	3
P ₃	2	5
P ₄	4	2

- 1 Draw the Gantt chart using FCFS.
- 2 Calculate CT, TAT, WT and RT for every process.
- 3 Calculate average WT and average TAT.
- 4 Repeat using non-preemptive SJF.
- 5 Explain why the results differ.

Analytical Question 2: SRTF

For:

$$P_1(0, 8), \quad P_2(1, 4), \quad P_3(2, 2), \quad P_4(3, 1)$$

where each tuple is (AT, BT) :

- 1 Construct the SRTF Gantt chart.
- 2 Identify every preemption.
- 3 Calculate CT, TAT, WT and RT.
- 4 Count the number of process switches.
- 5 Explain which process experiences the largest waiting time and why.

Higher-order thinking

How would the result change if context switching required 0.2 time units?

Analytical Question 3: Round Robin

Consider:

$$P_1(0, 7), \quad P_2(1, 4), \quad P_3(2, 3), \quad P_4(3, 2)$$

For $q = 2$:

- 1 Construct the Gantt chart.
- 2 Calculate CT, TAT, WT and RT.
- 3 Calculate the averages.
- 4 Repeat for $q = 4$.
- 5 Compare response time and number of context switches.

Design question

Why can increasing the quantum reduce overhead but make RR behave more like FCFS?

Analytical Question 4: Priority Scheduling

Processes:

Process	AT	BT	Priority
P ₁	0	6	3
P ₂	1	2	1
P ₃	2	4	4
P ₄	3	3	2

Assume smaller number = higher priority.

- 1 Solve using non-preemptive priority scheduling.
- 2 Calculate CT, TAT, WT and RT.
- 3 Identify any possible starvation scenario.
- 4 Explain how aging could reduce starvation.

Higher-Order Comparison Question

Suppose a system has:

- many short interactive jobs,
- a few long CPU-bound jobs,
- frequent user interaction,
- and a requirement for fair CPU access.

Compare:

FCFS, SJF, SRTF, RR, Priority

Discuss:

- expected response time,
- average waiting time,
- fairness,
- starvation risk,
- context-switch overhead,
- suitability for the workload.

Summary

- **FCFS**: simple, non-preemptive, but vulnerable to convoy effect.
- **SJF**: selects the shortest available job; good average waiting time under suitable assumptions.
- **SRTF**: preemptive SJF; reacts to newly arriving short jobs.
- **Round Robin**: time-sharing policy based on a time quantum; emphasizes responsiveness and fairness.
- **Priority**: schedules according to importance; may require aging to control starvation.

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

Gantt Chart → **CT/ST** → **TAT/WT/RT** → **Average** →
Interpretation

Key Takeaways for Exam Problems

- 1 Clearly identify AT and BT.
- 2 Determine whether the algorithm is preemptive.
- 3 For SRTF, recalculate remaining time whenever a relevant process arrives.
- 4 For RR, maintain the ready queue carefully.
- 5 For Priority, verify the priority convention.
- 6 Draw the Gantt chart before calculating metrics.
- 7 Use:

$$TAT = CT - AT, \quad WT = TAT - BT, \quad RT = ST - AT.$$

- 8 Interpret the numerical result rather than only reporting averages.